TIET LostLink: A Secure Campus Lost-and-Found System

Project Proposal for UCS503P
Submitted to: Nisha Thakur
Thapar Institute of Engineering and Technology

August 18, 2026

1 Higher-order goal

. . . secondary framing

This project aims to improve resource recovery, trust, and student convenience within the Thapar Institute of Engineering and Technology (TIET) campus.

Students frequently lose personal belongings such as bags, water bottles, wallets, keys, books, and electronic accessories. Information about these items is normally shared through different class groups, hostel groups, social-media stories, or informal communication.

The broader goal of the project is to reduce the number of usable personal items that remain unclaimed or must be unnecessarily replaced.

The immediate engineering objective is to create a simple, secure, and deployable web application that centralizes lost-and-found reports, suggests possible matches, supports ownership claims, and records the successful return of items.

The proposed application will assist students in finding possible matches. It will not automatically declare a person to be the legal owner of an item.

2 Time-to-value

. . . primary heuristic

- A working version of the main reporting and searching workflow will be developed during the first two weeks.
- The project will prioritize essential functionality over technically complex machine-learning features.
- A simple and explainable weighted matching algorithm will be used instead of training or deploying large artificial-intelligence models.
- The project will be developed through four weekly iterations.
- Each iteration will produce a working improvement that can be demonstrated and tested.
- A deployed minimum viable product will be completed by the end of the fourth week.

3 Problem Statement

Students who lose or find personal items on campus currently depend on fragmented communication channels such as class WhatsApp groups, hostel groups, social-media posts, informal word of mouth, and physical security desks.

The current process creates the following problems:

- **Fragmented information:** A lost-item report and a found-item report may be posted in different groups and may never reach the same students.
- **Poor searchability:** Students cannot easily search existing posts using item category, date, colour, brand, or campus location.
- **Low persistence:** Lost-and-found messages quickly disappear below newer conversations.
- **Repeated posts:** Students may need to share the same information in several groups.
- **Weak ownership verification:** If every identifying detail is displayed publicly, an unrelated student may use those details to make a false claim.
- **No structured handover process:** There is no consistent method for submitting claims, checking private information, approving a claim, and confirming that an item was returned.

Why this matters:

TIET has several academic blocks, hostels, activity areas, food outlets, and event venues. A centralized and mobile-friendly system can make it easier for students to report, search for, claim, and return lost items.

4 Proposed Solution *... software proposition*

4.1 Overview

We propose a web-based lost-and-found platform named **TIET LostLink**.

The platform will provide the following facilities:

- **Student authentication:** Students will register and log in using an institutional email address.
- **Lost-item reporting:** Students will submit the item category, description, date, approximate location, colour, brand, and an optional photograph.
- **Found-item reporting:** A finder will submit similar structured information while keeping certain identifying characteristics private.
- **Search and filtering:** Reports can be filtered by category, date, status, colour, brand, and location.
- **Possible-match suggestions:** A lightweight matching algorithm will compare lost and found reports using structured fields.
- **Ownership claims:** A student can submit private answers and supporting evidence to claim a found item.
- **Claim approval:** The finder can approve or reject the submitted claim.
- **Secure handover:** An approved claim will generate a one-time handover code.
- **Administrator controls:** An administrator can review flagged posts, disputed claims, and inappropriate activity.

4.2 Supported item classes

To keep the project achievable within four weeks, the first version will support a limited number of item categories.

The supported categories will be:

1. Backpacks and bags
2. Water bottles and tumblers
3. Wallets and purses
4. Earbuds and headphone cases
5. Keys and keychains
6. Books and notebooks

These items were selected because they often contain visible or describable characteristics such as:

- colour and pattern;
- brand;
- stickers;
- scratches;
- keychains;
- cover design; and
- other personal markings.

Cash, jewellery, government documents, plain charging cables, and visually identical mass-produced items will not be included in the initial automated matching workflow.

4.3 Core workflow

1. A student registers and logs into the application.
2. The student submits either a lost-item report or a found-item report.
3. The system saves the report and generates a unique report ID.
4. The system compares the report with reports of the opposite type belonging to the same item category.
5. Possible matches are ranked using category, brand, colour, date, location, and description similarity.
6. The student reviews the possible matches.
7. If a likely match is found, the student submits an ownership claim.
8. The finder reviews the private claim details and either approves or rejects the claim.
9. An approved claim generates a temporary handover code.
10. The students exchange the item and confirm the handover.
11. The report status is changed to `Returned`.

4.4 Ownership verification

The main challenge is that the actual owner may not always possess a receipt, serial number, or photograph of the lost item.

Therefore, the system will use a layered verification process rather than depending on one form of proof.

- When reporting a found item, the finder records one or more private characteristics that are not displayed publicly.
- The claimant must describe the item before these private details are revealed.
- Claim answers will be timestamped and cannot be modified after submission.
- Possible ownership evidence may include:
  - a hidden scratch or sticker;
  - the contents of a bag or wallet;
  - an older photograph;
  - the exact brand or model;
  - a device name;
  - unlocking or pairing an electronic device;
  - a matching key and lock; or
  - the approximate time and location at which it was lost.
- For a generic item with no reliable identifying information, the system will recommend supervised handover through an administrator or security desk.
- The system will assist with evidence collection but will not automatically determine legal ownership.

4.5 Operational constraints

- The complete project must be implemented by two students within four weeks.
- The application must work on mobile and desktop web browsers.
- The project will not depend on private TIET databases or WebKiosk integration.
- The application will not process payments.
- The application will not collect precise live location.
- The application will not require a separate mobile application.
- Matching will use a simple algorithm instead of a separate machine-learning service.
- Email notification may be added if time permits; in-application notifications will be sufficient for the initial version.

5 Solution Approach *... engineering focus*

The project will focus on software workflow, database correctness, privacy, security, and usability.

The matching algorithm will be simple, explainable, and implementable directly in the backend.

5.1 Possible-match algorithm *... lightweight and explainable*

The system will first apply mandatory filters:

- The report types must be opposite.
- One report must be `Lost` and the other must be `Found`.
- Both reports must belong to the same item category.
- Both reports must have an active status.

The remaining candidates will receive a weighted score.

Matching condition	**Score**
Same item category	30 points
Same brand	15 points
Same or similar colour	15 points
Same or nearby campus location	15 points
Dates within three days	15 points
Description keyword similarity	10 points
Maximum possible score	100 points

The match score can be expressed as:

$$M = C + B + R + L + D + T$$

where:

- C represents category matching;
- B represents brand matching;
- R represents colour matching;
- L represents location matching;
- D represents date matching; and
- T represents description keyword similarity.

An initial interpretation of the score will be:

- $M \geq 75$: high-likelihood candidate;
- $50 \leq M < 75$: possible candidate; and
- $M < 50$: weak candidate.

These labels will only assist users in reviewing reports. They will not prove ownership.

5.2 Description similarity

The backend will perform basic text processing:

1. Convert descriptions to lowercase.
2. Remove punctuation.
3. Split descriptions into individual keywords.
4. Remove common words such as “the”, “a”, “and”, and “near”.
5. Calculate the percentage of important keywords shared between the two descriptions.

   For example:

   Lost description: “Black Sony earbuds with a scratched case.”

   Found description: “Sony wireless earbuds in a black charging case.”

   The common meaningful words include:

   `sony, earbuds, black, case`

This provides a simple description-similarity score without requiring a large language model or a separate Python service.

5.3 Web architecture

A simple three-tier architecture will be used.

- **Frontend:** React and Vite will provide the student and administrator interfaces.
- **Backend:** Node.js and Express.js will implement the REST APIs and business rules.
- **Database and managed services:** Supabase will provide PostgreSQL, student authentication, and image storage.

The simplified architecture is:

`React Frontend` $\rightarrow$ `Express Backend` $\rightarrow$ `Supabase PostgreSQL`

Supabase Storage will store item and claim photographs.

5.4 Basic technology stack

The proposed technology stack is:

Component	Technology
Frontend	React with Vite
Styling	Basic CSS or Tailwind CSS
Routing	React Router
Backend	Node.js and Express.js
Database	Supabase PostgreSQL
Authentication	Supabase Authentication
Image storage	Supabase Storage
API communication	Axios or Fetch API
Backend testing	Jest and Supertest
API testing	Postman
Frontend deployment	Vercel
Backend deployment	Render
Version control	Git and GitHub
CI	Basic GitHub Actions workflow

This stack was selected because it:

- is commonly used and well documented;
- reduces authentication and image-storage development time;
- does not require maintaining a separate machine-learning server;
- can be deployed using free or low-cost services; and
- is realistic for a two-person team within four weeks.

5.5 Database entities

The initial database will contain the following tables:

- **Users:** Stores user ID, name, email, role, verification status, and account status.
- **ItemReports:** Stores lost and found reports, item properties, image reference, location, date, public description, private detail, and status.
- **Matches:** Stores suggested report pairs and calculated match scores.
- **Claims:** Stores claimant information, private claim answers, supporting evidence, and approval status.
- **Handovers:** Stores one-time code information and return confirmation.
- **Notifications:** Stores in-application notifications.
- **Flags:** Stores inappropriate-post and suspicious-user reports.

5.6 CI/CD and fast delivery

A basic GitHub Actions workflow will be used to:

- install project dependencies;
- run backend tests;

- check whether the frontend builds successfully; and
- prevent clearly broken code from being merged.

The team will use feature branches and pull requests so that each student can review the other student's code.

6 Evaluation Criterion

... measurable and attributable

The project will be evaluated using the accuracy of match suggestions, workflow completion, security, and user experience.

6.1 Primary evaluation metric

The primary metric will be **Top-five match suggestion success**.

A test will be considered successful if the correct corresponding report appears within the five highest-ranked match suggestions.

$$\text{Top-5 Success Rate} = \frac{\text{Test cases containing the correct report in the top five}}{\text{Total test cases}} \times 100$$

The initial target will be a top-five success rate of at least 75% on a controlled test set.

6.2 Secondary evaluation metrics

- **Search response time:** Search and filtering results should normally be displayed within two seconds during the pilot.
- **Claim completion rate:** Percentage of pilot users who successfully submit and review an ownership claim without assistance.
- **Handover completion rate:** Percentage of approved claims that successfully complete the handover workflow.
- **Security:** Unauthorized users must not be able to view private item details or claim evidence.
- **User clarity:** Pilot users will rate the clarity of the report, claim, and handover workflow on a scale of 1–5.
- **System reliability:** All critical backend and end-to-end test cases should pass before the final demonstration.

6.3 Pilot validation plan

- Recruit approximately 10–20 students for controlled testing.
- Create approximately 30–50 sample lost-and-found report pairs.
- Include visually and descriptively similar items to test false matches.
- Ask students to complete reporting, searching, claiming, and handover tasks.
- Record matching results, task completion, workflow time, and user feedback.

The pilot will use test or voluntarily submitted data. The project will not claim official university adoption unless formal approval is received.

7 Scalability *... with foresight*

The initial project will remain small, but the architecture will support future growth.

1. **Database scalability:**
   - Use pagination for report lists.
   - Add indexes on category, report type, date, status, and location.
   - Store image files separately from report data.
2. **Application scalability:**
   - Keep the frontend and backend separate.
   - Use stateless REST APIs.
   - Avoid recalculating matches unnecessarily.
3. **Future institutional scalability:**
   - More TIET hostels can be added.
   - New item categories can be introduced.
   - Other educational institutions can be supported using an `institution_id` field.

8 Engine availability heuristic

A mature set of tools is available to implement the project without building complex infrastructure.

- React for a responsive web interface.
- Node.js and Express.js for REST APIs.
- Supabase for authentication, PostgreSQL, and image storage.
- Jest and Supertest for backend testing.
- Postman for manual API testing.
- Vercel and Render for deployment.
- GitHub for version control, pull requests, and basic CI.

These tools allow the team to focus on the lost-and-found workflow, ownership verification, security, and usability.

9 Project scope and deliverables *... iteration-friendly*

9.1 Initial deliverable *... first iteration*

- Student registration and login
- Lost-item reporting form
- Found-item reporting form

- Image upload
- Report feed
- Search and filters
- Unique report IDs
- Report status
- Initial deployed application

9.2 Subsequent deliverables

- Weighted possible-match suggestions
- Public and private detail separation
- Ownership claim submission
- Claim approval and rejection
- One-time handover code
- Return confirmation
- In-application notifications
- Administrator moderation
- Automated tests
- Final deployment
- Project documentation

10 Four-week implementation plan

Week	Student 1	Student 2
1	Database design, Supabase setup, authentication, and Express backend setup	Wireframes, React setup, login interface, navigation, and report-form interface
2	Lost/found report APIs, search, filters, and weighted matching algorithm	Report feed, report details, image upload, search, and filtering interface
3	Claim APIs, authorization rules, handover code, notifications, and administrator APIs	Claim interface, finder review screen, handover interface, notifications, and administrator dashboard
4	Backend testing, security testing, bug fixing, and backend deployment	Frontend testing, usability testing, documentation, and frontend deployment

Both students will work together on:

- requirements and UML diagrams;

- API integration;
- pull-request reviews;
- end-to-end testing;
- user testing;
- final documentation; and
- project demonstration.

11 Risks and mitigations

- **Insufficient ownership proof:** Use hidden details, sealed claim answers, supporting evidence, and supervised handover for uncertain cases.
- **False ownership claims:** Limit repeated claims, record rejected claims, and allow administrator review.
- **Generic items:** Do not automatically approve claims for items that cannot be distinguished reliably.
- **Incorrect match suggestions:** Present suggestions as possible matches and require human review.
- **Private information exposure:** Restrict private details to the claimant, finder, and administrator.
- **Malicious image upload:** Restrict image type and size and store uploads using Supabase Storage.
- **Project scope becomes too large:** Complete reporting, searching, claiming, and handover before adding optional features.
- **Low pilot adoption:** Use controlled test cases with a small group of students.
- **Deployment difficulties:** Deploy a basic version during the first or second week instead of waiting until the end.

12 Future enhancements

The following features may be added after the four-week project:

- Image-similarity matching using a pretrained model
- Semantic description matching
- Email notifications
- QR-based handover
- A native mobile application
- Hostel or security-desk integration
- Advanced analytics

- Support for additional item categories
- Multi-institution support

These features are intentionally excluded from the core implementation to keep the first version feasible.

13 Summary

TIET LostLink is a web-based platform for centralizing campus lost-and-found reports, suggesting possible matches, collecting private ownership information, and supporting secure handover.

The project uses a basic and achievable technology stack consisting of:

- React;
- Node.js;
- Express.js;
- Supabase PostgreSQL;
- Supabase Authentication;
- Supabase Storage; and
- GitHub.

The initial system will not depend on a complex machine-learning pipeline. It will use a weighted and explainable matching algorithm based on category, brand, colour, date, location, and description keywords.

This approach keeps the project realistic for two students working within a maximum period of four weeks while still demonstrating:

- requirements analysis;
- frontend and backend development;
- relational database design;
- authentication and authorization;
- secure claim verification;
- matching algorithms;
- software testing;
- Agile teamwork;
- version control; and
- deployment.

The final result will be a complete, deployable, and testable lost-and-found application rather than an unfinished system with unnecessary technical complexity.